

4. Binary Search with Big-O Analysis

Aim

To search for an element in a sorted array using the **Binary Search algorithm** and analyze its Big-O complexity.

Algorithm

1. Read a sorted array.
2. Set $low = 0$ and $high = n - 1$.
3. Find the middle element.
4. If the key is greater, search the right half.
5. Otherwise, search the left half.
6. Repeat until the element is found or the search range becomes empty.

Procedure

1. Start the program.
2. Read the number of elements.
3. Read the elements in sorted order.
4. Read the element to be searched.
5. Calculate the middle position.
6. Compare the middle element with the search element.
7. Display whether the element is found.

main.py	Visualize	Run ▶	Output
<pre>1 n = int(input("Enter number of elements: ")) 2 a = list(map(int, input("Enter sorted elements: ").split())) 3 4 key = int(input("Enter element to search: ")) 5 6 low = 0 7 high = n - 1 8 found = False 9 10 while low <= high: 11 mid = (low + high) // 2 12 13 if a[mid] == key: 14 print("Element found at position:", mid + 1) 15 found = True 16 break 17 elif a[mid] < key: 18 low = mid + 1 19 else: 20 high = mid - 1 21 22 if not found: 23 print("Element not found")</pre>			<pre>Enter number of elements: 5 Enter sorted elements: 1 4 8 6 5 Enter element to search: 3 Element not found</pre>

Result

Thus, the required element was successfully searched using Binary Search, and its worst-case time complexity was determined as **$O(\log n)$** .